Diffusion — Study Sheet

Phases A & B. Theory first, then the questions, my answers, and the corrections.

PHASE A — What a DDPM actually is

1 · The goal hasn't changed

```
Data:    x₀ ~ P_{x₀}              (data distribution)
Goal:    learn to sample from P_{x₀}
```

Identical to the VAE goal. Everything in diffusion serves this one sentence.

2 · Where we came from

```
        Encoder             Decoder
x ──► z ──► x̂
   one bottleneck, compressed (k ≪ d)
```

3 · The leap: a chain of latents (h-VAE)

```
ENCODING   x ▶ z₁ ▶ z₂ ▶ z₃ ▶ … ▶ z_T
DECODING   z_T ▶ z_{T−1} ▶ … ▶ x

A DDPM is an h-VAE. But the latents are NOT more abstract —
they are MORE DESTROYED. A hierarchy of damage, not of abstraction.
```

4 · The three properties that make it a DDPM

(a) Multiple latent spaces
`z₁ … z_T`

(b) Same size as data
`dim(z_t) = dim(x)`

(c) Encoder is **FIXED**
not learnable

Property **(c)** is the deep one. Your notes (p73):

```
VAE  : qφ(z|x) : φ is a NN that learned
DDPM: q(z|x)  : is FIXED & not learnable
```

→ Diffusion removes one of VAE's two unknowns entirely. There is no forward-side posterior to approximate — the forward process is arithmetic you define and control.

5 · The notation trap

```
VAE:    x₁, x₂, …, xₙ     subscript = WHICH data point  (n examples)
DDPM:   x₀, x₁, …, x_T     subscript = HOW MUCH NOISE     (T versions of ONE)

x₀ ─noise─► x₁ ─noise─► x₂ ─► … ─► x_T
image      slightly     more              pure
           corrupted    corrupted         static
```

Same symbol, different meaning. **T is a hyperparameter** (~1000 in practice).

6 · What is learned vs what is fixed

	VAE	Diffusion (DDPM)
latents	one, z	T of them, x₁…x_T
latent dimension	k ≪ d (compressed)	k = d (same size)
encoder	**learned** qφ(z|x)	**FIXED**
q is...	unknown → approximated	known exactly
decoder	learned Pθ(x|z)	learned Pθ(x_{t−1}|x_t)
ELBO	2 terms	**3 terms**
sampling	1 pass from N(0,I)	**T sequential passes**

```
LEARNED  :  the REVERSE chain   x_T → x_{T−1} → … → x₀
FIXED    :  the FORWARD chain   x₀ → x₁ → … → x_T

Train by destroying. Generate by un-destroying.
```

7 · Why the bottleneck moved from size to noise

```
VAE bottleneck   =  TOO FEW NUMBERS   (k ≪ d — information cannot fit)
DDPM bottleneck  =  TOO MUCH NOISE    (information is destroyed)

t = 0      ᾱ_t = 1.0    →  the image, exact      100% signal
t = T/2    ᾱ_t ~ 0.5    →  half signal, half static
t = T      ᾱ_t ~ 0      →  x_T ≈ ε               0% signal
```

At t=T, x_T carries **no information** about x_0 — it is statistically independent of it. So the model cannot copy; it must **invent** what was destroyed, and it can only invent from what it has learned about the data distribution. That is the learning pressure VAE got from a narrow z.

One-line summary. A DDPM is a hierarchical VAE with **T** latents the same size as the data, where the encoder is **fixed arithmetic** instead of a learned network — so the only thing you learn is the reverse (denoising) chain, and the only thing you generate with is that same chain run backwards.

8 · The questions, my answers, and the corrections

Q1 **What does the subscript mean in $x_1 \ldots x_n$ (VAE) vs $x_0 \ldots x_T$ (DDPM) — and what is x_3 in each?**

MY ANSWER

In VAE all x1...xn are data points, but in DDPM x0 is data and other are latent variable at its respective T.

✓ CORRECT

COMPLETE ANSWER

The x_3 comparison, spelled out:

VAE: x_3 = the **3rd image** in the dataset — a completely different picture from x_1.
DDPM: x_3 = the **same single image** as x_0, after 3 rounds of noise.

So $x_1 \to x_2$ in VAE means *different example*; in DDPM it means *same example, more damage*.

Q2 **DDPM has `dim(x_t) = dim(x₀)`, yet VAE insists `k ≪ d`. Why no compressed latent? What plays the bottleneck's role instead?**

MY ANSWER

(not attempted)

— SKIPPED

ANSWER

VAE's bottleneck is **dimensional**: you cannot fit 784 numbers into 32, so the model must discard information and keep only what matters.

DDPM keeps full size, so the bottleneck must come from elsewhere: **the noise**. At `t = T`, x_T is essentially ε — the information is *gone*, not merely squeezed. So the model still cannot copy; it must supply the missing structure from what it learned.

And it is arguably *better*: VAE has **one** bottleneck level, DDPM has **T** — a smooth ramp from 100% signal to 0%, giving T distinct training difficulties instead of one.

Q3 **"DDPM: only decoding is learned." Which VAE machinery does that let us skip — and why was it needed in VAE?**

MY ANSWER

the fied encoder facilittes the DDPM to skip encoder mechanism of VAE

⚠ RIGHT INSTINCT — NEEDED PRECISION

CORRECTED

Not just "skip the encoder". What is actually skipped:

```
a LEARNED encoder qφ(z|x)      → no encoder network at all
the variational APPROXIMATION   → q is KNOWN exactly (fixed arithmetic)
the ELBO gap on the forward side → q is dictated, not chosen
```

In VAE you had to *invent* q because the true posterior was intractable, then live with a loose bound. In DDPM there is nothing to invent — the forward process **is** the q.

Q4 **True or false: *"x_T is a compact, abstract representation of x_0, like z in a VAE."***

MY ANSWER

False , in vae it was latent represntation of x , but in DDPM its noisy

✓ CORRECT CONCLUSION — ONE WORD TO SHARPEN

SHARPER

"Noisy" suggests the information is still there, just hidden. **It isn't.**

```
"noisy"    →  information present but obscured
REALITY    →  information GONE

q(x_T) ~ N(0,I)  REGARDLESS of which x₀ you started from
                 ∴ knowing x_T tells you NOTHING about x₀
```

Better phrasing: *"In VAE, z is a compressed semantic representation of x. In DDPM, x_T is pure noise, statistically independent of x_0 — it represents nothing at all."*

Q2', In one sentence each: what is VAE's bottleneck made of, and what is DDPM's made of?

MY ANSWER

VAE's bottleneck made of single hard compressin from x -> z , but DDPM is made of T steps . This helps DDPM denoising the noisy image .

⚠ RIGHT SHAPE — WRONG SUBSTANCE

CORRECTED

You described the **granularity**, not the **substance**:

```
VAE bottleneck   =  TOO FEW NUMBERS    (k ≪ d)
DDPM bottleneck  =  TOO MUCH NOISE     (information destroyed)
```

"T steps" is a genuine bonus (more granularity, T training signals instead of 1) — but it is not *what the constraint is*. The constraint is that at t=T the signal is zero.

Q3', VAE needed the reparameterization trick to make the encoder trainable. DDPM also samples randomly (ε_t), yet never needs it. Why not?

MY ANSWER

the reparameterization is not skipped but the term isalray their i equation

✓ CORRECT

COMPLETED — THE PRECISE REASON

```
VAE :  z ~ qφ(z|x)     distribution PARAMETERISED BY φ
       ∇φ E_q[ · ]    → cannot move ∇ inside: q depends on φ
                       ∴ needed the trick to expose μ, σ

DDPM:  ε ~ N(0, I)       INDEPENDENT of θ
       ∇θ E_ε[ · ]    → CAN move ∇ inside: ε does not depend on θ
                       ∴ nothing to fix
```

The forward equation $x_t = \sqrt{\alpha_t}\, x_{t-1} + \sqrt{(1-\alpha_t)}\varepsilon_t$ **is already** the reparameterized form — mean + std × external noise. The solution derived in VAE's Phase D is simply *given* here.

Key distinction: in VAE the randomness sits **between the parameters and the loss**. In DDPM it sits **upstream of everything**, baked into the input. x_t is an *input* to the network, not an output of it — so $\nabla\theta$ never passes through a random draw.

Q5 If DDPM keeps $\dim(x_t) = \dim(x_0)$, why doesn't it just learn the identity and copy the input?

MY ANSWER

to make the DDPM learn we create that noisy image at X_t just so that it doesnt copy the same input to produces output at x_t

✓ CORRECT

SHARPENED

Right — and the reason is stronger than "can't copy". Without noise the task would be $x_t = x_0 \to \hat{x} = x_0$: learn the identity, learn nothing.

The noise is not merely a blocker — **it is what creates the learning signal.** The corrupted parts of x_t are *gone*. To restore them the model must **invent** them, and it can only invent from what it has learned about how data looks.

Same function as VAE's bottleneck: **destroy information so the model must supply it from learned knowledge rather than passing it through.**

Phase B — Forward: the fixed destruction

Source p74–75. The shortest phase in the topic — **nothing is learned here.**

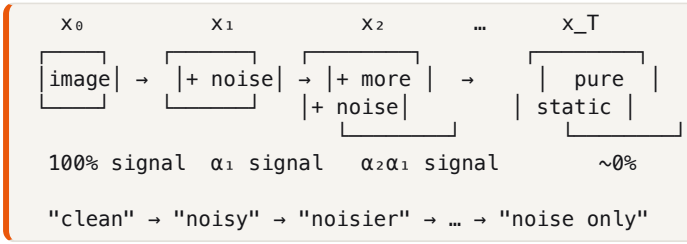
B1 · The equation

```
x_t  =  √α_t · x_{t−1}  +  √(1 − α_t) · ε_t ,      ε_t ~ N(0, I)
        ─── signal ───      ──────── new noise ────────

x₁ = √α₁ x₀ + √(1−α₁) ε₁
x₂ = √α₂ x₁ + √(1−α₂) ε₂
   ⋮
x_T = √α_T x_{T−1} + √(1−α_T) ε_T
```

And the key constraint (p74): **$\alpha_1 \ldots \alpha_T$ are fixed scalars in $[0,1]$.** Chosen by us. Not learned. Not optimised.

B2 · What actually happens



Each step mixes in a **fresh** noise draw. $\varepsilon_1 \neq \varepsilon_2 \neq \varepsilon_3$ — never reused.

B3 · The Markov property

```
x_t  ⊥⊥  x_{t−2} … x₀  |  x_{t−1}
```

Read as: **once you know x_{t-1}, everything earlier is irrelevant.** No memory beyond one step. This is what later lets you factorise $q(x_{1:T}|x_0) = \prod q(x_t|x_{t-1})$ — a product instead of an intractable joint.

B4 · So each step is a simple Gaussian

```
q(x_t | x_{t−1})  =  N( x_t ;  √α_t · x_{t−1} ,  (1 − α_t) I )
                              ──────┬──────     ──────┬──────
                                  mean              variance
```

You can write this down in closed form. That is the entire point — no network, no ELBO, no approximation. Just q.

B5 · Why exactly $\sqrt{\alpha}$ and $\sqrt{(1-\alpha)}$?

They are chosen so that **variance is preserved**:

```
Var(x_t) = Var( √α_t x_{t−1} + √(1−α_t) ε_t )
         = α_t · Var(x_{t−1})  +  (1−α_t) · 1        (independent parts add; Var(ε)=1)

if Var(x_{t−1}) = 1 :   = α_t + (1 − α_t)  =  1     ✓
```

So **every** x_t has unit variance, including x_T. That is precisely why your notes can claim:

> **$q(x_T) \sim N(0, I)$ — the stationary distribution of the Markov chain.**
>
> The schedule is **engineered so that $N(0,I)$ is the fixed point** of the destruction process. Two consequences: (1) the final latent distribution is known exactly, so sampling starts from $N(0,I)$ with no guesswork; (2) information decays **smoothly** from 100% to ~0%, giving T graded difficulty levels.

B6 · What is NOT in this phase

No loss. No ELBO. No gradients. No network. If this phase felt anticlimactic — **good, it should.** The forward process is the setup, deliberately trivial so that training has a target with a known, exact answer.

B7 · Preview: the closed-form shortcut

```
q(x_t | x₀) = N( √ā_t · x₀ ,  (1 − ā_t) I )        where  ā_t = ∏_{i=1..t} α_i
```

Your notes derive this on p85–86 (Phase E). It is what makes training practical: pick a random t and construct x_t **immediately**, with no loop.

Phase B — the questions, my answers, and the corrections

B-Q1 **Write the forward step `x_t` in terms of `x_{t−1}`, and say which symbol is (a) fixed by you, (b) drawn fresh each step.**

MY ANSWER (1ST ATTEMPT)

I cant write the full equation , but the fixed part is second term and first part where we have x t_1 is which is calculated at each t step

✕ THE SPLIT WAS BACKWARDS

CORRECTED

```
x_t  =  √α_t · x_{t−1}  +  √(1−α_t) · ε_t
        ──┬──   ──┬──      ────┬────   ┬
      fixed    carried    fixed    DRAWN FRESH
```

The error: "the second term is fixed" — but `ε_t` lives in the second term, and it is the one random thing. The alphas are fixed in *both* terms.

B-Q1' **Retry: which symbol is fixed, which is drawn fresh, which is neither?**

MY ANSWER

the aplha terms are fixed , the x_t-1 is inherited from last steo and epislon term i s random noise added in each step different

✓ CORRECT — ALL THREE

CONFIRMED

Exactly right now: **α** fixed, **x_{t-1}** inherited, **ε_t** fresh and different every step. This is the foundation the rest of diffusion is built on — good that it is clean now.

B-Q2 **Where exactly does the `(1−α_t)I` come from? Show the reasoning, don't just quote the formula.**

MY ANSWER

it is to adjust the variance , if x_t-1 is 1 then variance become unit . so to have variance 1 we add the alpha -1 term

✓ CORRECT CONCEPT

TIGHTENED

```
Var(x_t) = α_t · Var(x_{t−1})  +  (1−α_t) · 1
                                  ────┬────
                    if Var(x_{t−1}) = 1 :
        = α_t + (1 − α_t)  =  1     ✓
```

Why `(1−α_t)` specifically: `α_t` is already used up scaling the signal. Whatever fraction of variance the signal keeps (`α_t`), the noise must receive exactly the remainder to bring the total back to 1 — two halves of a budget that must sum to one.
Minor: the term is `(1−α_t)`, not "alpha−1".

B-Q3 **Prove `Var(x_{t−1})=1 ⇒ Var(x_t)=1`, then explain why that lets the notes claim `q(x_T) ~ N(0,I)`.**

MY ANSWER (1ST ATTEMPT)

BECAUSE X_T VARIANCE IS 1 THEN WE CAN SAY q(x_t) as N(0,I)

⚠ HALF RIGHT — VARIANCE IS NOT ENOUGH

CORRECTED

`N(0,I)` needs **two** things: mean **0** *and* variance **1**. You covered only the variance.

```
q(x_t) = N( √ᾱ_t · x₀ ,  (1 − ᾱ_t) I )
          ────┬────      ────┬────
            MEAN           VARIANCE

at t = T,  ᾱ_T ≈ 0 :
   mean     = √0 · x₀  =  0      ✓
   variance = 1 − 0       =  1      ✓
```

Why the mean matters more than it seems: suppose variance were 1 but the mean were still √ᾱ_T x₀ with ᾱ_T not small. Then x_T would still be *centred on a scaled copy of* x_0 — you could read the image off the mean. It is the **vanishing mean** that destroys the information; unit variance merely makes it *standard*.

B-Q3' **Name both conditions for `q(x_T) ~ N(0,I)` — and which single quantity causes both.**

MY ANSWER

first means should go to zero , then only the variance becomes 1 . whcih makes N(0,I)

⚠ BOTH CONDITIONS NAMED — CAUSALITY WRONG

CORRECTED

You named both conditions correctly. But **"first…then" is wrong** — they are not sequential, and the mean going to zero does not *cause* the variance to become 1.

```
BOTH happen at once, from ONE cause:   ᾱ_T ≈ 0

   ᾱ_T ≈ 0   →   mean     = √ᾱ_T x₀  → 0     (signal term dies)
   ᾱ_T ≈ 0   →   variance = 1 − ᾱ_T   → 1     (noise takes full budget)
```

The single quantity is ᾱ_T (the cumulative product of alphas). It does both jobs simultaneously — kills the signal *and* hands the full variance to the noise.

Correct phrasing: *"ᾱ_T ≈ 0 causes both: the mean → 0 and the variance → 1. Together that is `N(0,I)`."*

B-Q4 **The notes say α₁…α_T are *"fixed scalars"*. What would go wrong if you learned them like network weights?**

MY ANSWER

(didn't understand)

— NEEDED THE ANSWER

ANSWER — IT WOULD LET THE MODEL CHEAT

If α_t were learnable, gradient descent would push α_t → 1:

```
α_t = 1   ⇒   x_t = √1 · x_{t−1} + √0 · ε_t = x_{t−1}
          ⇒   x_t = x_{t−1} = … = x₀        ← NO NOISE AT ALL

task becomes:   input x_t = x₀   →   predict x₀   →   output x₀
```

Learning α would let the model choose **"add no noise"** — making training trivially easy (the identity function) and completely useless. **The noise is what CREATES the learning signal. If you let the model tune the noise away, it will.**

Three more reasons (for completeness):

① **The ELBO assumes q is fixed.** A learned q returns you to full variational inference — losing the entire Phase A advantage.

② **Training needs x_t in closed form.** You construct x_t yourself to get a free supervised target. If α moved, the target drifts every step.

③ **It is a design decision, not a data parameter.** α encodes *"how fast do I want to destroy the signal"* — tuned for sample quality. It is not a property of the data distribution.

Phase C — Reverse: the model itself

Source p76. Short phase — but this is the first one where we meet **the model** rather than the setup.

C1 · The model is a factorisation

```
Pθ(x₀, x₁, …, x_T)  =  P(x_T) · ∏_{t=1}^{T} Pθ(x_{t-1} | x_t)
                        ──┬──   ──────┬──────
                      starting     each reverse
                     distribution      step

start  →  x_T  →  x_{T-1}  →  …  →  x₁  →  x₀
       (noise)                          (image)
```

This joint IS "the diffusion model." When someone says "a diffusion model", this product is what they mean.

C2 · One step, in detail

```
Pθ(x_{t-1} | x_t)  =  N( x_{t-1} ;  μθ(x_t) , Σθ(x_t) )
                                    ──┬──   ──┬──
                                   learned   learned
```

Your notes (p76): *"**μθ(x_t) & Σθ(x_t) are learned via ELBO.**"*

C3 · ⚠ The direction trap

```
FORWARD  (q, FIXED):      q(x_t | x_{t-1})       cleaner → noisier
                            ↑       ↑
                          output   input

REVERSE  (Pθ, LEARNED):   Pθ(x_{t-1} | x_t)     noisier → cleaner
                            ↑       ↑
                          output   input
```

at t = 500	question asked
forward q	"given x_{499}, what does x_{500} look like?"
reverse Pθ	"given x_{500}, what was x_{499}?"

Same two letters. **Opposite arrow.** The vertical bar flipped sides — the input/output roles swapped.

If you ever feel lost in diffusion, **check which way the bar points.** Half the confusion evaporates.

C4 · What is fixed vs what is learned

```
P(x_T)  =  N(0, I)                ← FIXED. Not learned. Note: NO θ.
                                      Known because the forward chain ends there.

Pθ(x_{t-1}|x_t) = N(μθ, Σθ)       ← LEARNED. This is ALL of the training.

model  =  T learned transitions  +  1 fixed starting distribution
```

The starting distribution costs nothing because **Phase B engineered it**: the variance-preserving schedule makes `N(0,I)` the **stationary distribution** of the forward chain. The forward process **manufactured your starting distribution for free** — which is why diffusion can sample from nothing.

Notational note: the notes write `Pθ(x_T)` in the factorisation but then `P(x_T) = N(0,I)` with no θ. The intent is that it is fixed, not learned.

C5 · The VAE parallel

```
VAE:    Pθ(x, z)      =  Pθ(x | z)      · P(z)
                          decoder         prior

DDPM:   Pθ(x₀, x₁…x_T) =  Pθ(x₀|x₁) · Pθ(x₁|x₂) · … · P(x_T)
                          ↑  T decoders, chained
```

VAE has **one** decode step z → x. DDPM has **T** decode steps. That is the entire structural difference.

C6 · Why is the reverse Markov? (be careful)

Your notes say (p76): *"arises because of the **Markovian (1st order) assumption in the reverse direction**…"* — note the word **assumption**.

```
FORWARD:   x_t depends ONLY on x_{t-1}        ✓ genuinely Markov (by construction)

REVERSE:   x_{t-1} depends on x_t AND x₀
           q(x_{t-1}|x_t) = ∫ q(x_{t-1}|x_t,x₀) · q(x₀|x_t) dx₀
                            ── tractable ──   ── the problem ──
```

Many different x_0 can produce similar x_t, so knowing x_t does not tell you which one it was. Reversing a Markov chain does **not** give a Markov chain.

Analogy: heat spreading through metal is perfectly Markov going forward. But you cannot reverse it from one frame — *"the metal is warm in the middle"* doesn't tell you whether it started warm on the left or the right.

Consequence: `Pθ(x_{t−1}|x_t)` is an **approximation**, not the true reverse. Its cost appears in Phase D's **consistency term**, which compares our Markov model against the honest `q(x_{t−1}|x_t,x₀)`.

C7 · One network, T uses

```
✗  T separate networks, one per timestep   →  T × parameters. Absurd.
✓  ONE network, reused at every step, with t as an INPUT

model =  εθ(x_t, t)      or     μθ(x_t, t)
              ↑
       the second argument is essential
```

Why t must be an input: the *same weights* must behave completely differently at

```
t = 900   →  almost pure noise, must invent most of the image
t = 5     →  barely noisy, only a small nudge needed
```

One network cannot do both unless it knows **which timestep** it is in. That is why Phase F needs **positional embeddings of t** (p91).

Phase C in one line. The model is the product `P(x_T)∏Pθ(x_{t−1}|x_t)` — a fixed `N(0,I)` start times T learned Gaussian reverse steps, each conditioning on the *noisier* variable and producing the *cleaner* one, all sharing one t-conditioned network.

Phase C — the questions, my answers, and the corrections

C-Q1 **Write the DDPM joint as a product. Which factor(s) are learned, which is fixed — and why does the fixed one cost nothing?**

MY ANSWER

The distrbution we got from output of encoder p_0(x_t) is fixed here and the product of p_0(x_t-1 | x t) is what would be changing

✕ "ENCODER" IS THE WRONG INSTINCT

CORRECTED

① **There is no encoder in the reverse process.** In VAE, z came out of a *learned network*. Here x_T is simply `N(0,I)` — nothing computes it.

```
GENERATIVE MODEL:   x_T ~ N(0, I)                  ← sampled directly. No network.
TRAINING DATA:      x₀ ←forward q→ x_T                ← fixed arithmetic. No network.
                    (q has NO learned parameters at all)
```

② **The symbol is `P(x_T)`, not `Pθ(x_t)`.** The missing θ *is* the point — it signals "not learned".

```
Pθ(x₀…x_T) = P(x_T) · ∏_{t=1..T} Pθ(x_{t−1}|x_t)
              ──┬──   ──────┬──────
          FIXED=N(0,I)   LEARNED (T factors)
```

Why it costs nothing: Phase B engineered the schedule so `N(0,I)` is the **stationary distribution** of the forward chain. The endpoint is `N(0,I)` *by construction* — nothing to learn or approximate. **`The forward process manufactured your starting distribution for free.`**

C-Q1' **Retry: (a) where does x_T come from, (b) which factor is fixed/learned, (c) why is the fixed factor free?**

MY ANSWER

a. x_T is encoders output , b. first term p0(x_T) second is learned c. fixed factor are alphas

✕ SAME ERROR REPEATED — PLUS A MISREAD OF (c)

CORRECTED — ALL THREE

(a) ✕ Still the encoder. **No network produces x_T**, in either context (see the calc box in C-Q1). VAE's encoder was a *learned network with weights ϕ*; diffusion has nothing like that.

(b) ⚠ Right shape, two fixes: it is `P(x_T)` with **no θ**; and there are **T** learned factors, not one.

(c) ✕ The alphas *are* fixed, but they are **schedule parameters**, not a factor in the joint. The fixed **factor** is `P(x_T) = N(0,I)`. And the reason it is free is the **stationary-distribution** property from Phase B — not the alphas themselves.

C-Q2 **Compare `q(x_t|x_{t-1})` with `Pθ(x_{t-1}|x_t)`: which goes cleaner→noisier, which is learned, and what changes about the bar?**

MY ANSWER

q(x_t|x_t-1) gors from cleanear to noiser , the P_0 is what learned , it changes what direction its working

✓ CORRECT ON ALL THREE

SHARPENED (C)

```
q(x_t | x_{t-1})      INPUT (right of bar) = the CLEANER one  → output noisier
                      FORWARD, fixed

Pθ(x_{t-1} | x_t)    INPUT (right of bar) = the NOISIER one  → output cleaner
                      REVERSE, learned
```

The bar keeps its position visually, but **the input/output roles swap**. Same index pair `(t-1, t)`, arrow running in opposite directions.

C-Q3 **A colleague says: *"The reverse is Markov because the forward is — reversibility preserves the property."* Correct?**

MY ANSWER

(not attempted)

— ANSWERED

ANSWER — FALSE, AND IT IS THE KEY SUBTLETY

Reversing a Markov chain does not give a Markov chain.

```
FORWARD:  x_t depends ONLY on x_{t-1}        ✓ genuinely Markov
REVERSE:  x_{t-1} depends on x_t AND x₀
          q(x_{t-1}|x_t) = ∫ q(x_{t-1}|x_t,x₀) q(x₀|x_t) dx₀
```

Many different x_0 produce similar x_t, so x_t under-determines where you came from.

Analogy: heat spreading through metal is Markov forward, but one frame does not tell you whether it started warm on the left or the right.

So $P_\theta(x_{t-1}|x_t)$ is an **approximation**. Its cost appears in Phase D's **consistency term**, which compares it against the honest $q(x_{t-1}|x_t,x_0)$.

C-Q4 **Why does θ appear in every transition, yet we need not use T separate networks? What must the network receive as input?**

MY ANSWER

(not attempted)

— ANSWERED

ANSWER

```
one network:   εθ(x_t, t)    or    μθ(x_t, t)
                     ↑
            t is an INPUT, not a parameter
```

```
t = 900  →  almost pure noise, must invent most of the image
t = 5    →  barely noisy, only a small nudge needed
```

One set of weights cannot do both unless it knows **which timestep** it is in. Hence `t` is fed in — and Phase F gives it a **sinusoidal positional embedding** (p91).

θ is one number-set; t is the switch that reconfigures it for each of the T steps.